

Orders Stack

(1 sec, 256mb)

ข้อนี้ขอเปลี่ยน CP::stack<T> ของเราเป็น Orders Stack โดยขอนิยาม Orders Stack คือ Stack ที่มีค่าไม่เพิ่มขึ้น จาก Bottom ไป Top ของ Stack ขอยกตัวอย่างดังนี้ โดยให้ด้านขวาสุด เป็น Top of stack

- [1, 2, 3, 4] ไม่เป็น Orders Stack
- [4, 3, 2, 1] เป็น Orders Stack
- [99, 10, 2, -5, -5] เป็น Orders Stack

โดยให้เพิ่มและแก้ไขบริการต่อไปนี้

- void push(const T& value) เป็นการเพิ่มข้อมูลเข้าไปใน stack โดยต้องคงความเป็น Orders Stack ไว้เหมือนเดิม
- void pop() เป็นการลบข้อมูลออกจากด้านบนสุดของ stack

ตัวอย่างเช่น หากเริ่มต้นจาก stack ว่าง แล้ว push(5), push(9) และ push(40) ใส่ stack ตามลำดับ จะทำให้ stack ของเรามีข้อมูลเป็น [40, 9, 5] (Top of stack อยู่ด้านขวา) จะสังเกตว่า 9 และ 40 จะถูกดันลงไปอยู่ Bottom of stack ตามลำดับ หาก push(99) และ pop() เพิ่ม จะทำให้ stack มีข้อมูลเป็น [99, 40, 9] (Top of stack อยู่ด้านขวา)

(มีต่อหน้าถัดไป)

ข้อบังคับ

- โจทย์ข้อนี้จะมีไฟล์ตั้งต้นมาให้ ซึ่งประกอบด้วยไฟล์ stack.h, main.cpp และ student.h อยู่ ให้นิสิตเขียน code เพิ่มเติมลงในไฟล์ student.h เท่านั้น และการส่งไฟล์เข้าสู่ระบบ grader ให้ส่งเฉพาะไฟล์ student.h เท่านั้น
 - สามารถ include library เพิ่มเติมหากต้องการ
 - ไฟล์ student.h จะต้องไม่ทำการอ่านเขียนข้อมูลใด ๆ ไปยังหน้าจอหรือคีย์บอร์ดหรือไฟล์ใด ๆ
 - หากใช้ VS Code ให้ทำการ compile ที่ไฟล์ main.cpp
- ** main ที่ใช้จริงใน grader นั้นจะแตกต่างจาก main ที่ได้รับในไฟล์โปรเจกต์เริ่มต้นแต่จะทำการทดสอบในลักษณะเดียวกัน ****

คำอธิบายฟังก์ชัน main

main() จะเป็นการทดลองใช้งาน stack โดย main จะอ่านข้อมูลนำเข้าดังนี้

- บรรทัดแรก เป็นจำนวนเต็ม T ระบุจำนวนรอบของการทดสอบ สำหรับแต่ละรอบการทดสอบ

จากนั้น main ทำการสร้าง `CP::stack<int> stk` ขึ้นมา และจะรับข้อมูลนำเข้าแบบต่อเนื่องจนกว่าจะเจอคำสั่ง "q" โดยมีรูปแบบคำสั่งที่เป็นไปได้ทั้งหมดดังต่อไปนี้

- si หมายถึงให้พิมพ์ค่าของ `stk.size()` ออกทางจอ
- to หมายถึงให้พิมพ์ค่าของ `stk.top()` ออกทางจอ
- pu X หมายถึงให้เรียก `stk.push(X)`
- po หมายถึงให้เรียก `stk.pop()`
- q หมายถึงให้ main หยุดการทำงาน

เมื่อออกจากการทำงานในแต่ละรอบของการทดสอบ โปรแกรมจะแสดง จำนวนข้อมูล, capacity และข้อมูลทุกตัวใน stk ตามลำดับ

ชุดข้อมูลทดสอบ

รับประกันว่าจำนวนครั้งที่มีการเรียก operation ต่าง ๆ ใน stack จะรวมกันในทุกรอบการทดสอบแล้วไม่เกิน 500,000 ครั้ง นอกจากนี้จะไม่มีการเรียก top หรือ pop เมื่อ state ของ stack ที่ถูกต้อนั้นว่างอยู่

- 10% push ข้อมูลที่น้อยกว่าหรือเท่ากับ top แน่ ๆ (push ปกติ) และ ไม่เรียก pop
- 10% push ข้อมูลที่น้อยกว่าหรือเท่ากับ top แน่ ๆ (push ปกติ)
- 15% `mCap = 1`
- 15% stack เก็บข้อมูลเป็น int
- 20% มีการเรียก operation ของ stack รวมกันในการทดสอบแล้วไม่เกิน 5,000 ครั้ง
- 30% ไม่มีเงื่อนไขเพิ่มเติม

ตัวอย่างการทำงานของ main

ข้อมูลนำเข้า	ข้อมูลส่งออก
1 pu 4 pu 99 pu 6 si to po to q	3 4 6 Stack Size: 2 Stack Capacity: 4 Data: 99 6 -----
2 pu 3 pu 4 pu 5 to q pu 50 to pu 20 pu 80 to pu 10000 to q	3 Stack Size: 3 Stack Capacity: 4 Data: 5 4 3 ----- 50 20 20 Stack Size: 4 Stack Capacity: 4 Data: 10000 80 50 20 -----